

D1 — DPE SYNCHRONOUS EXECUTION + PERFORMANCE AUDIT

Repository depth_perception_engine v1.2.0

Branch release/v1.2.0 · **HEAD** f4ce645

Scope Audit + measurement only — production code unchanged

Baseline / final regression 983 passed, 0 failed, 0 skipped

D1 DPE SYNCHRONOUS EXECUTION AUDIT: PASS — READY FOR DPE/HPE CONTRACT DECISION

Clean rerun. Every finding and measurement was re-derived from the current working tree; no state or conclusion from the lost prior session was reused.

Raw machine-readable results: [benchmarks/d1_execution/results/d1_execution_audit.json](#)

Contents

1. Repository / version / environment
2. Baseline regression
3. Authoritative DPE runtime graph
4. Public interface classification
5. StereoObservation contract
6. Observation identity vs coordinate-frame identity — CRITICAL
7. Timestamp semantics
8. GeometryFrame contract
9. Mutable-state inventory
10. Thread-safety / reentrancy conclusion
11. Construction / resource reuse
12. Reset / close semantics
13. Motion / IMU path
14. Calibration / physical-unit contract
15. Main synchronous benchmark
16. Stage-by-stage benchmark
17. Temporal vs non-temporal benchmark
18. Resolution scaling
19. Repeated-run stability
20. Frame-order / drop / discontinuity behavior — CRITICAL for HPE backpressure
21. Failure semantics
22. Native / Python compute characterization
23. DPE vs frozen NPE
24. Theoretical provider-overlap opportunity
25. CPU-contention risks
26. Exact HPE-facing DPE contract
27. Ten orchestration-readiness answers
28. HPE blocker classification
29. Files created / changed
30. Final regression
31. Final recommendation

Clean rerun. A previous D1 session was lost before its report could be preserved. Every finding and every number below was re-derived from the current working tree in this session. The prior session's untracked artefacts under [benchmarks/d1_execution/](#) were moved out of the repository to the session scratchpad **before** any measurement began, so nothing in this report inherits that session's state or conclusions.

Scope. Audit and measurement only. No production source file was created, edited, or deleted. No asynchronous execution, queue, thread, future, worker, result buffer, multiprocessing, shared memory, [async / await](#), CUDA, TensorRT, or ROS dependency was added. HPE, NPE, and [mp01_perception](#) were not touched. Nothing was staged, committed, pushed, or tagged.

1. Repository / version / environment

Item	Value
git branch --show-current	release/v1.2.0
git rev-parse HEAD	f4ce645e73bef01930a59b3ed7d68c4adda1b165
git status --short (start)	?? benchmarks/d1_execution/
git tag --list	level3-e1-frozen, level4-temporal-perception-frozen, v1.0.0, v1.0.1, v1.1.0, v1.1.1
git describe --always --tags	v1.1.1-2-gf4ce645
Package version	depth_perception_engine 1.2.0
Python	3.13.14 (CPython)
NumPy	2.5.1
OpenCV	5.0.0
cv2.getNumThreads()	8
cv2.useOptimized()	True
CPU	11th Gen Intel Core i5-1135G7 @ 2.40 GHz — 4 cores / 8 threads, max 4.20 GHz
CPU governor	powersave
RAM	7.4 GiB total
OS / kernel	Kali GNU/Linux Rolling, Linux 7.0.12+kali-amd64
Calibration fixture	examples/config/stereo_calibration.xml (real hardware calibration)
Calibration image size	320 × 240
Benchmark fixtures	benchmarks/i1_stereo_accuracy/fixtures.py low-frequency-canvas + disparity-remap technique, via benchmarks/i6_temporal/fixtures.py

Active (qualified) configuration. The full V1 candidate `PipelineConfig` already frozen in `benchmarks/i0_baseline/scenarios.py::latency_scenario` — reused verbatim rather than reinvented:

```
enable_geometry, enable_obstacle_geometry, enable_free_space_rays,
enable_surface_geometry, enable_boundary_geometry, enable_opening_geometry,
enable_temporal, enable_temporal_stabilization, enable_rotation_compensation,
enable_motion_aware_reliability, enable_temporal_persistence,
enable_geometry_frame                                = True
temporal_gap_limit_s = 5.0, temporal_max_age_s = 100.0, temporal_max_records = 30
rectify = True
body_T_camera_left = RigidTransform(I, [0.05, 0.0, 0.02] m, camera_optical_left -> body)
```

Measurement-quality disclosure

This is a shared developer laptop and it was **not idle**. During an early benchmark pass the 1-minute load average reached **19.4** on a 4C/8T machine (CLion 108 % CPU, Chrome 81 %, Rider 31 %, PyCharm 17 %), which inflated the observed median to 47.8 ms with a 249 ms maximum. The authoritative run below was taken at a load average of **1.02 → 2.44**, and every latency figure is reported three ways: pooled (as observed), per-trial medians across 5 independent repeats, and a contention-robust estimator (minimum / p10 / best trial median). External preemption can only ever *add* time, so the contention-robust figures are lower bounds on true cost, and the pooled median is an upper bound under realistic desktop load. CPU pinning was tested and rejected — `taskset -c 0-3` made results *worse* (40.8 ms vs 37.5 ms) because it starves OpenCV's 8-thread SGBM region.

2. Baseline regression

Run before any other work, with the unmodified tree:

```
.venv/bin/python -m pytest -q
983 passed, 4 warnings in 21.33s
```

	Count
passed	983
failed	0
skipped	0
errors	0
warnings	4
runtime	21.33 s

The 4 warnings are pre-existing and unrelated to D1: two `RuntimeWarning: invalid value encountered in cast` pairs from `temporal/rotation_compensation.py:223-224`, emitted by `tests/test_d1l_degradation_validation.py`'s deliberate NaN/Inf angular-velocity degradation tests. Baseline is green; the audit proceeded in full.

3. Authoritative DPE runtime graph

The expected path was **verified, not assumed**. `process_geometry_frame()` is confirmed to be a thin composition over the single geometry implementation:

```
process_geometry_frame(observation)      pipeline.py:969
  -> process_observation(observation)    pipeline.py:1004 (called exactly once)
  -> result.geometry_frame if already built (enable_geometry_frame=True)
      else _build_geometry_frame(result) pipeline.py:1007
```

`process_observation()` is the sole geometry implementation. There is no second pipeline and no standalone/embedded mode flag. The complete ordered stage graph, as it actually executes under the qualified configuration:

```
StereoObservation
|
| 0 destructure fields                                pipeline.py:397-402
|   (left/right image, left/right timestamp, motion_hint, motion_hints)
|   NOTE: observation.calibration and observation.frame_id are NOT read
| 1 closed-pipeline guard -> RuntimeError             pipeline.py:403
| 2 stereo pair validation require_matching_stereo_pair() pipeline.py:408
| 3 t0 = perf_counter(); result_timestamp = left or right pipeline.py:409-414
| 4 rectification RectificationEngine.rectify()        pipeline.py:435 [gated: rectify=True]
| 5 grayscale     cv2.cvtColor(BGR2GRAY)              pipeline.py:441
| 6 disparity     DisparityEngine.compute_disparity()  pipeline.py:443
| 7 depth        DepthEstimator.estimate()            pipeline.py:446
| 8 reliability   compute_shadow_zone_mask()          pipeline.py:461 [gated]
| 9 point cloud  PointCloudBuilder.build()            pipeline.py:486 [gated: enable_geometry]
|10 body transform transform_point_cloud()            pipeline.py:509 [gated: body_T_camera_left]
|11 obstacle geom build_obstacle_cloud()              pipeline.py:539 [gated]
|12 free space   build_free_space_rays()              pipeline.py:553 [gated]
|13 geometry metrics build_geometry_metrics()         pipeline.py:563
|14 surface      build_surface_evidence()              pipeline.py:576 [gated]
|15 boundary     build_boundary_evidence()             pipeline.py:603 [gated]
|16 opening      build_opening_evidence()              pipeline.py:635 [gated]
|17 traversability SceneInterpreter.analyze/decide()   pipeline.py:649-651
|18 clearance mask compute_ramp_zone_mask() | shadow mask pipeline.py:665-677
|19 clearance     ThreatAssessor.assess()              pipeline.py:679
|20 temporal admission TemporalHistory.admit()         pipeline.py:736 [gated: enable_temporal]
|21 rotation compensation compute_rotation_compensation() pipeline.py:792 [gated; admitted frames only]
|22 temporal consistency compute_temporal_consistency() pipeline.py:807 [admitted frames only]
|23 stabilization compute_temporal_stabilization()    pipeline.py:835 [gated; admitted frames only]
|24 motion-aware reliability compute_motion_aware_reliability() pipeline.py:871
|   [gated; runs even on REJECTED frames]
|25 persistence   TemporalPersistenceTracker.update()  pipeline.py:907
|   [gated; runs even on REJECTED frames]
|26 elapsed_ms; build_result()                        pipeline.py:924-942
|27 GeometryFrame assembly build_geometry_frame()      pipeline.py:960 / 1023
|28 runtime bookkeeping _frames_processed / _last_*    pipeline.py:962-964
|   -> GeometryFrame
```

Every requested stage was located; **no stage was invented**. Two ordering facts matter for HPE:

- Stages 24 (motion-aware reliability) and 25 (persistence) run **outside** the "was this frame admitted?" block. A chronology-rejected frame therefore still mutates persistence tracker state.
- Stage 27 only reads fields off the already-built result — the GeometryFrame path recomputes nothing (measured at 0.066 ms, section 16).

4. Public interface classification

Interface	Classification	Notes
<code>depth_perception_engine.core</code>	HPE-APPROPRIATE	The declared embedded boundary. Re-exports the same objects as the package root; imports nothing from <code>standalone</code> (structurally enforced by <code>tests/test_dual_interface_architecture.py::TestStandaloneOptionality</code>).
<code>DepthPerceptionPipeline</code> (class)	HPE-APPROPRIATE	Construct once, reuse.
<code>DepthPerceptionPipeline.process_geometry_frame(obs)</code>	HPE-APPROPRIATE — THE ONE RUNTIME METHOD	<code>StereoObservation</code> → <code>GeometryFrame</code> . Never returns <code>None</code> .
<code>DepthPerceptionPipeline.process_observation(obs)</code>	INTERNAL / LEGACY COMPATIBILITY	The single geometry implementation, but returns the legacy <code>DepthPerceptionResult</code> . HPE should not consume that shape.
<code>DepthPerceptionPipeline.process(left, right, ...)</code>	LEGACY COMPATIBILITY	Loose-argument adapter; builds a <code>StereoObservation</code> and delegates. Used by <code>mp01_perception</code> , examples, and the test suite.
<code>DepthPerceptionPipeline.from_config(...)</code>	HPE-APPROPRIATE (optional)	Identical to the constructor.
<code>StandaloneStereoInterface</code>	STANDALONE ONLY / NOT FOR HPE	Sensor-facing adapter: combined-frame splitting, calibration file loading, motion-hint building. Lazily resolved via module <code>__getattr__</code> so importing the package never loads it.
<code>load_stereo_calibration(path)</code>	STANDALONE ONLY	Deliberately not re-exported from <code>core</code> . An embedded consumer supplies a <code>StereoCalibration</code> object it already owns.
<code>pipeline.api</code> Tier-2 helpers (<code>process_stereo_pair</code> , <code>compute_disparity</code> , <code>estimate_depth</code> , <code>classify_traversability</code> , <code>detect_obstacles</code>)	NOT FOR HPE	Stateless one-shot helpers. <code>process_stereo_pair</code> builds a fresh pipeline per call and therefore has no cross-frame temporal state.
<code>reset()</code>	HPE-APPROPRIATE (lifecycle)	Complete temporal amnesia. See section 12.
<code>close()</code>	HPE-APPROPRIATE (lifecycle)	One-way terminal state; idempotent.
<code>health()</code>	HPE-APPROPRIATE (observability)	Lifecycle snapshot only, never a per-frame diagnosis. Still readable after <code>close()</code> .
<code>.config</code> / <code>.calibration</code> / <code>.temporal_history</code> properties	INTERNAL / diagnostics	Read-only exposure; <code>temporal_history</code> leaks a Level-4 internal type and should not appear in an HPE contract.

The one runtime DPE method HPE should invoke:

```
DepthPerceptionPipeline.process_geometry_frame(observation: StereoObservation) -> GeometryFrame
```

5. StereoObservation contract

`models/result.py:30` — `@dataclass(frozen=True, slots=True)`, 8 fields. Images are held **by reference**; constructing one copies nothing.

5. StereoObservation contract

Field	Type	Req.	Semantics	Ownership	Read by DPE?	Consumed at	Reaches GeometryFrame?
<code>left_image</code>	<code>np.ndarray</code>	yes	Left stereo image, BGR or grayscale	by reference, never copied	yes	rectify → grayscale → SGBM	indirectly (rasters derive from it)
<code>right_image</code>	<code>np.ndarray</code>	yes	Right stereo image	by reference, never copied	yes	rectify → SGBM	indirectly
<code>left_timestamp</code>	<code>Optional[float]</code>	no	Opaque caller-defined float	value	yes	<code>result_timestamp</code> (wins over right)	yes → <code>GeometryFrame.timestamp</code>
<code>right_timestamp</code>	<code>Optional[float]</code>	no	Fallback timestamp	value	yes	<code>result_timestamp</code> fallback only	only when <code>left_timestamp</code> is <code>None</code>
<code>calibration</code>	<code>Optional[StereoCalibration]</code>	no	Reserved for future multi-rig use	reference	NO	nowhere	no
<code>frame_id</code>	<code>Optional[str]</code>	no	Documented as observation identity	value	NO	nowhere	no
<code>motion_hint</code>	<code>Optional[MotionHint]</code>	no	Single sample for this capture	reference	yes (association only)	<code>TemporalRecord.motion_hint</code> ; contents never integrated	no
<code>motion_hints</code>	<code>Optional[Sequence[MotionHint]]</code>	no	Bounded window over the inter-frame interval	reference	yes	rotation compensation, motion-aware reliability, persistence	indirectly, via status/state fields

Two fields are declared but non-functional

`process_observation()` destructures exactly six fields (`pipeline.py:397-402`). `calibration` and `frame_id` are never read. This is documented behaviour, not an accident — `process_observation()` 's own docstring says "*observation.calibration / .frame_id remain reserved and unread*". A repository-wide scan for `observation.frame_id / obs.frame_id` across all of `src/depth_perception_engine/` returned **zero matches**.

Direct probe — `StereoObservation(frame_id="observation-X")` → `process_geometry_frame()` → recursive scan of the entire returned `GeometryFrame` object graph (depth 4, all nested evidence types, dicts and lists) for the literal string "`observation-X`" :

```
observation_id_found_anywhere_in_geometry_frame : []      (zero hits)
observation_id_found_in_legacy_result           : []      (zero hits)
production_source_reads_of_observation_frame_id : []      (zero hits)
```

6. Observation identity vs coordinate-frame identity — CRITICAL

The two concepts are cleanly separable today, and DPE currently implements **only the second**.

A. Observation / transaction identity — "GeometryFrame X and SemanticFrame X came from the same HPEObservation." **DPE has no such field on its output.**

B. Coordinate-frame identity — "which coordinate system is this data expressed in." This is what every `frame_id` in DPE means, without exception:

Field	Observed value	Meaning
<code>StereoObservation.frame_id</code>	"observation-X" (caller-set)	Nothing. Never read; inert.
<code>GeometryFrame.frame_id</code>	"camera_optical_left"	Coordinate frame. Hardcoded at <code>fusion/result_builder.py:457</code> as <code>FrameId.CAMERA_OPTICAL_LEFT</code> .
<code>PointCloud.frame_id</code> (geometry)	"camera_optical_left"	Coordinate frame
<code>geometry_body.frame_id</code>	"body"	Coordinate frame
<code>obstacle_cloud.frame_id</code>	"body"	Coordinate frame
<code>free_space_rays.frame_id</code>	"body"	Coordinate frame
<code>surface_evidence[i].frame_id</code>	"body"	Coordinate frame
<code>boundary_evidence[i].frame_id</code>	"camera_optical_left"	Coordinate frame
<code>clearance_evidence[i].frame_id</code>	"camera_optical_left"	Coordinate frame
<code>region_evidence["TL"].frame_id</code>	"camera_optical_left"	Coordinate frame

`geometry/provider.py` 's own module docstring is explicit: "`frame_id` is the frame every top-level metric field on this object ... is expressed in — always `frames.FrameId.CAMERA_OPTICAL_LEFT`".

The four required answers

1. **Is `observation-X` preserved?** — **NO.** It appears nowhere in `GeometryFrame`, nowhere in `DepthPerceptionResult`, and is never read by any production source line.
2. **Is `GeometryFrame.frame_id` an observation ID or a coordinate-frame ID?** — Unambiguously a **coordinate-frame ID**. It is a compile-time constant of the pipeline, identical on every frame.
3. **Is there any other authoritative `GeometryFrame` field carrying immutable observation identity?** — **NO.** The only other provenance-bearing field is `timestamp`, which is a caller-supplied, unvalidated, non-unique float (section 7).
4. **Can HPE perform an exact ID-based DPE/NPE join using only the public `GeometryFrame` contract?** — **NO.**

P0 — HPE-INTEGRATION CONTRACT DEFECT

DPE's authoritative output contract carries no observation transaction identity. HPE's planned exact same-observation join of `GeometryFrame X` with `SemanticFrame X` cannot be expressed against the current public `GeometryFrame` contract. The only available correlator is `timestamp`, a caller-supplied float that DPE neither validates, normalizes, nor guarantees unique.

Not fixed in D1, as instructed.

Recommended shape (stated, not implemented). `GeometryFrame.frame_id` must **not** be repurposed — it carries real, load-bearing coordinate-frame semantics that the whole nested evidence type graph depends on. DPE needs **two distinct concepts**, conceptually:

`observation_id` immutable, opaque, caller-supplied transaction identity, copied verbatim from `StereoObservation` onto `GeometryFrame`, never interpreted by DPE `coordinate_frame` the existing `frame_id` semantics, unchanged

The input side already has the field (`StereoObservation.frame_id`) — it is simply inert. Whether that field is renamed for clarity, or a parallel `observation_id` is introduced on both types, is a contract decision for the DPE/HPE boundary, not a D1 decision.

7. Timestamp semantics

Question	Answer
Source of <code>GeometryFrame.timestamp</code>	<code>result.timestamp</code> , set from <code>result_timestamp</code> (pipeline.py:414), passed through by <code>build_geometry_frame</code> (result_builder.py:456)
Left-vs-right precedence	<code>left_timestamp</code> wins whenever it is not <code>None</code> . Measured: left=10.0, right=99.0 → <code>GeometryFrame.timestamp</code> = 10.0
Fallback	left=None, right=42.0 → 42.0
Neither supplied	<code>GeometryFrame.timestamp</code> = <code>None</code> ; the frame is still produced
Accepted type / unit	Any float. No unit conversion and no unit validation anywhere in DPE.
Are units explicitly defined?	No. Docstrings call it "opaque, caller-defined".
De facto unit	Seconds , whenever <code>enable_temporal=True</code> — <code>temporal_max_age_s</code> and <code>temporal_gap_limit_s</code> are compared directly against raw timestamp differences. Nothing enforces this.
Monotonicity enforced?	For temporal history only . Geometry output is produced regardless.
Duplicates accepted?	No → <code>REJECTED_DUPLICATE_TIMESTAMP</code> ; the existing record is kept unchanged.
Decreasing accepted?	No → <code>REJECTED_OLDER_TIMESTAMP</code> ; history never reordered or mutated.
NaN / <code>None</code>	→ <code>REJECTED_INVALID_TIMESTAMP</code> . The invalid value is still passed through onto <code>GeometryFrame.timestamp</code> .
Sufficient as a unique exact-join key?	NO

Measured chronology probe (single instance, qualified config):

submitted	admission status	<code>GeometryFrame.timestamp</code>	history len	consistency
1.0	ACCEPTED	1.0	1	INSUFFICIENT_EVIDENCE
2.0	ACCEPTED	2.0	2	CONSISTENT
2.0	REJECTED_DUPLICATE_TIMESTAMP	2.0	2	None
1.5	REJECTED_OLDER_TIMESTAMP	1.5	2	None
3.0	ACCEPTED	3.0	3	CONSISTENT
NaN	REJECTED_INVALID_TIMESTAMP	NaN	3	None
None	REJECTED_INVALID_TIMESTAMP	None	3	None
4.0	ACCEPTED	4.0	4	CONSISTENT

Conclusion. Timestamp should be **secondary validation around an immutable observation ID**, not the primary join key. It fails as a join key on three independent grounds: it is optional (legally `None`), it is not guaranteed unique (DPE rejects a duplicate from *history* but still returns a `GeometryFrame` carrying that duplicate value), and it is caller-defined with no enforced unit — two providers fed from the same capture could legitimately stamp it differently. Semantics unchanged by D1.

8. GeometryFrame contract

22 fields. Presence measured under both the qualified configuration and the bare default `PipelineConfig(enable_geometry_frame=True)`.

Group	Field	Classification	Qualified	Default
identity/provenance	timestamp	optional (caller-supplied)	✓	✓
identity/provenance	frame_id	mandatory — always <code>camera_optical_left</code>	✓	✓
raster geometry	disparity_map	mandatory, (240, 320)	✓	✓
raster geometry	depth_map	mandatory, (240, 320)	✓	✓
raster geometry	valid_disparity_mask	mandatory in practice	✓	✓
raster geometry	valid_depth_mask	mandatory in practice	✓	✓
3-D geometry	geometry	gated: <code>enable_geometry</code>	✓	—
3-D geometry	geometry_body	gated: <code>enable_geometry</code> and <code>body_T_camera_left</code>	✓	—
obstacle/free-space	obstacle_cloud	gated: <code>geometry_body</code> + <code>enable_obstacle_geometry</code>	✓	—
obstacle/free-space	free_space_rays	gated: <code>geometry_body</code> + <code>enable_free_space_rays</code>	✓	—
quality	geometry_metrics	gated: <code>geometry_body</code>	✓	—
temporal	temporal_consistency	temporally derived	✓	—
temporal	temporal_stabilization	temporally derived, gated	✓	—
temporal	rotation_compensation_status	temporally derived, gated	✓	—
temporal	motion_aware_reliability	temporally derived, gated	✓	—
temporal	temporal_persistence	temporally derived, gated (3 flags)	✓	—
legacy-derived	region_evidence	always populated (9 regions)	✓	✓
clearance	clearance_evidence	always populated (20 beams)	✓	✓
surface	surface_evidence	gated (9 cells)	✓	—
boundary	boundary_evidence	gated (12 adjacencies)	✓	—
opening	opening_evidence	gated; positive-findings-only (len 0 observed)	✓	—
quality	quality	always populated when the frame is built	✓	✓

How HPE must interpret absence

Signal	Meaning	HPE must NOT infer
None on a gated field	The capability was never enabled or has no input. Nothing was evaluated.	That the evidence was evaluated and found absent.
Empty collection on <code>boundary_evidence</code> / <code>surface_evidence</code> / <code>region_evidence</code> / <code>clearance_evidence</code>	These are <i>always-one-record-per-cell</i> contracts. An empty list means the grid itself was empty — anomalous.	—
Empty opening_evidence list	"Evaluated; no opening confirmed this frame." This is the one positive-findings-only list.	That openings were not evaluated.
INSUFFICIENT_EVIDENCE state	Evaluated; not enough support to decide.	That the scene is clear, or that the answer is negative.
DEGRADED (in quality)	Evaluated; evidence exists but is below the healthy threshold. Still usable with reduced trust.	That the frame is invalid or should be dropped.
Unavailable capability (field None + corresponding <code>quality.*_state</code> is None)	The dimension was not assessed at all.	Any state whatsoever.

The absent-vs-degraded distinction is explicit in `GeometryFrameQuality`: a dimension reads `None` when the underlying capability is unavailable, and a real state string when it was assessed. Measured example under the qualified configuration:

```
overall_state          : DEGRADED
geometry_validity_state : DEGRADED
temporal_consistency_state: VALID
motion_reliability_state : VALID
persistence_state      : VALID
degradation_reasons     : ["GEOMETRY_VALIDITY:DEGRADED"]
```

Critically, HPE must not infer free space, clearance, or safety from absence. DPE never fabricates negative evidence: `UNKNOWN` stays `UNKNOWN`, and a missing measurement on either side of a boundary pair is never reinterpreted as a discontinuity.

9. Mutable-state inventory

18 instance attributes on one `DepthPerceptionPipeline`. Identity of every attribute was captured before and after 5 frames.

9. Mutable-state inventory

Attribute	Class	Purpose	Lifetime	Mutated per frame?	Frame-to-frame semantics depend on it?	Reset behaviour	Thread-safety consequence
<code>_config</code>	A	Tunable thresholds	construction	no	no	retained (same object)	safe to share
<code>_calibration</code>	A	Frozen calibration	construction	no	no	retained (same object)	safe to share
<code>_rectify</code>	A	Flag	construction	no	no	retained	safe
<code>_body_T_camera_left</code>	A	Frozen extrinsic	construction	no	no	retained	safe
<code>_rectifier</code>	A	RectificationEngine; holds <code>_map1_left/</code> <code>_map2_left/_map1_right/_map2_right</code>	construction	no (read-only)	no	retained (same object)	read-only; safe
<code>_disparity_engine</code>	A	Wraps <code>cv2.StereoSGBM</code> in <code>_stereo</code>	construction	no Python-visible mutation	no	retained (same object)	cv2.StereoSGBM owns internal scratch buffers — NOT safe to call concurrently
<code>_depth_estimator</code>	A	Holds Q; pure per frame	construction	no	no	retained	safe
<code>_point_cloud_builder</code>	A	Holds Q; allocates fresh arrays	construction	no	no	retained	safe
<code>_scene_interpreter</code>	A	Grid params only	construction	no	no	retained	safe
<code>_rectified_focal_length_px</code>	A	Derived scalar	construction	no	no	retained	safe
<code>_rectified_principal_point_px</code>	A	Derived scalar pair	construction	no	no	retained	safe
<code>_threat_assessor</code>	B	Per-beam EMA + debounce (<code>_ema_dist</code> , <code>_pending_count</code> , <code>_pending_status</code> , <code>_stable_status</code>)	per-instance	YES	YES	rebuilt fresh (new object id)	race
<code>_temporal_history</code>	B	Bounded TemporalRecord chronology	per-instance	YES	YES	<code>.clear()</code>	race
<code>_temporal_persistence_tracker</code>	B	Per-cell <code>_support_count</code> , <code>_absence_streak</code> , <code>_tracked_depth</code> , <code>_first_observed_timestamp</code> , <code>_last_state_grid</code> , <code>_grid_shape</code>	per-instance	YES	YES	<code>.clear()</code>	race
<code>_closed</code>	C	Lifecycle flag	per-instance	no	no	retained	benign
<code>_frames_processed</code>	C	Counter	per-instance	YES (rebound)	no	zeroed	lost update
<code>_last_confidence</code>	C	Bookkeeping	per-instance	YES (rebound)	no	None	lost update

Attribute	Class	Purpose	Lifetime	Mutated per frame?	Frame-to-frame semantics depend on it?	Reset behaviour	Thread-safety consequence
<code>_last_processing_time_ms</code>	C	Bookkeeping	per-instance	YES (rebound)	no	None	lost update

Class D (other): none found. No caches, deques, rolling windows, previous-frame image buffers, previous depth/disparity arrays, or scratch arrays are held on the pipeline. The one "previous frame" concept — `previous_latest` — is a *local variable* captured from `TemporalHistory.latest` before `admit()` runs, not persistent state.

Measured: only the three Class-C attributes were rebound across 5 frames. Every Class-A and Class-B object retained identical `id()`, confirming construct-once-reuse-forever.

10. Thread-safety / reentrancy conclusion

Classification: NOT THREAD-SAFE / ONE ACTIVE CALL AT A TIME .

No thread was created by this audit. The conclusion rests on observed shared-state mutation and static inspection.

Concrete evidence.

1. **Every frame mutates shared instance state.** Measured over 4 consecutive frames on one instance:

frame	ThreatAssessor._ema_dist sum	_pending_count sum	history len	persistence support sum
0	0.000 → 6.203	0 → 12	0 → 1	— → 369
1	6.203 → 6.203	12 → 24	1 → 2	369 → 738
2	6.203 → 6.203	24 → 0 (debounce fired)	2 → 3	738 → 1107
3	6.203 → 6.203	0 → 0	3 → 4	1107 → 1476

`all_frames_mutated_shared_state = True`. Two concurrent calls would interleave read-modify-write on `_ema_dist`, `_pending_count`, `_records`, and `_support_count`, corrupting EMA smoothing, debounce counting, chronology ordering, and per-cell persistence support.

1. **No synchronization exists.** A scan of every production `.py` file under `src/depth_perception_engine/` for `Lock`, `RLock`, `Semaphore`, `Condition`, `threading`, `asyncio`, `Queue`, `Executor` returned **zero matches**. Scan for `import threading|queue|asyncio|multiprocessing|concurrent` returned **zero matches**. There is no internal serialization to fall back on.
2. **TemporalHistory.admit() is a read-modify-write on a shared list.** It reads `self._records[-1].timestamp`, decides, then possibly `clear()`s and `append()`s (`history.py:158-171`). Two threads interleaving here can both observe the same "newest" record and both append, producing a non-monotonic buffer that violates the module's own stated invariant.
3. **The `cv2.StereoSGBM` matcher object is shared.** `_disparity_engine._stereo` is one OpenCV object reused on every call; it owns internal scratch buffers. Concurrent `compute()` on a single matcher instance is not supported by OpenCV.
4. **Runtime bookkeeping is a lost-update race.** `_frames_processed += 1` is not atomic.
5. **`reset()` / `close()` have no guard.** `reset()` rebinds `_threat_assessor` while another in-flight call may already hold a reference to the old one; `close()` flips `_closed` with no barrier.

Conclusion for HPE. Yes — but this is a *derived* conclusion, not an assumption. HPE should treat DPE as **one constructed pipeline instance + one active invocation at a time**. Concurrency, if ever wanted, must be either (a) one pipeline instance per worker, each with its own independent temporal state, or (b) external serialization around one shared instance. DPE itself must not grow a lock — that would be DPE owning execution policy, which the architecture reserves for HPE.

11. Construction / resource reuse

Construction creates: rectification maps (`cv2.initUndistortRectifyMap` → 4 arrays, only when `rectify=True`), the `cv2.StereoSGBM` matcher, `DepthEstimator` (holds `Q`), `PointCloudBuilder` (holds `Q`), `SceneInterpreter`, `ThreatAssessor` (EMA/debounce arrays), `TemporalHistory`, `TemporalPersistenceTracker`, and the derived rectified focal length / principal point scalars.

Measurement (n=40, after warmup)	mean	median	P95	P99
Construction, <code>rectify=True</code>	0.159 ms	0.157 ms	0.185 ms	0.186 ms
Construction, <code>rectify=False</code>	0.015 ms	0.018 ms	0.020 ms	0.021 ms
Rectification-map build (difference)	—	≈ 0.139 ms	—	—

Construction is **0.157 ms — about 0.5 % of a single frame's cost (31 ms)** and is excluded from every latency figure in this report.

Proof of reuse. Object `id()` captured before and after 20 frames on one instance:

<code>_rectifier</code>	stable ✓	<code>_threat_assessor</code>	stable ✓
<code>_disparity_engine</code>	stable ✓	<code>_temporal_history</code>	stable ✓
<code>_depth_estimator</code>	stable ✓	<code>_temporal_persistence_tracker</code>	stable ✓
<code>_point_cloud_builder</code>	stable ✓	SGBM matcher object	stable ✓
<code>_scene_interpreter</code>	stable ✓	all 4 rectification maps	stable ✓

Independently confirmed over **400** frames in section 19. `process_geometry_frame()` **reconstructs neither the pipeline nor any major persistent resource.** Verified.

12. Reset / close semantics

reset() — measured before/after on a pipeline with 5 frames of accumulated state

Cleared	Retained (same object)
<code>_threat_assessor</code> — rebuilt , new object id	<code>_rectifier</code> (identical id)
EMA state: <code>_ema_dist</code> sum 6.203 → 0.0; nonzero cells 12 → 0	<code>_disparity_engine</code> (identical id)
<code>_temporal_history</code> : len 5 → 0	<code>_calibration</code> (identical id)
Persistence: support sum 1845 → <i>None</i> , grid shape (60, 80) → <i>None</i>	<code>_config</code> (identical id)
<code>_frames_processed</code> : 5 → 0	rectification maps
<code>_last_confidence</code> : 0.6258 → <i>None</i>	
<code>_last_processing_time_ms</code> : set → <i>None</i>	

Debounce state is cleared as part of the `ThreatAssessor` rebuild (`_pending_count` / `_pending_status` / `_stable_status` come back fresh). Stabilization holds no persistent state of its own — it is recomputed per frame from `depth_snapshot_m` and the previous record, so clearing history clears it. Calibration, config, rectifier, and matcher are untouched by design.

close()

Call after <code>close()</code>	Result
<code>process_geometry_frame()</code>	RuntimeError — " <i>called after close() — construct a new pipeline instead of reusing a closed one.</i> "
<code>process_observation()</code>	RuntimeError (same)
<code>process()</code>	RuntimeError (same)
<code>reset()</code>	RuntimeError — " <i>reset() called after close().</i> "
<code>health()</code>	OK — still readable
<code>close()</code> again	OK — idempotent

`close()` is a one-way terminal state. It holds no hardware or file handles today; it establishes a real lifecycle contract rather than leaving post-close behaviour undefined.

When should HPE call `reset()` ? (reasoning only — no behaviour modified)

Situation	Call <code>reset()</code> ?	Why
Normal dropped observation	NO	History keys on timestamp, not sequence. A gap within <code>temporal_gap_limit_s</code> is absorbed correctly; beyond it, DPE self-restarts via <code>ACCEPTED_NEW_SEQUENCE</code> . Resetting would needlessly destroy valid EMA and persistence support.
Rejected DPE/NPE pair	NO	The DPE side is still chronologically valid. Discarding the <i>joined</i> result is HPE's concern; DPE's temporal chain remains sound.
Queue overload / backpressure drop	NO	Same as a dropped observation. Section 20 shows skipping is safe.
Sensor restart	YES	Scene continuity is broken and timestamps may restart. EMA/debounce/persistence carry-over would be actively wrong.
Calibration replacement	construct a NEW pipeline	<code>reset()</code> explicitly retains calibration and rectification maps. There is no supported way to swap calibration on a live instance.
Timestamp discontinuity	NO	DPE already handles this itself: a forward jump beyond <code>temporal_gap_limit_s</code> triggers <code>ACCEPTED_NEW_SEQUENCE</code> , which clears history <i>and</i> persistence (<code>pipeline.py:750-754</code>). Measured in section 20.
Mission restart	YES	Deliberate scene cut; complete amnesia is the correct semantics.

13. Motion / IMU path

Question	Answer
Which is authoritative?	<code>motion_hints</code> (the sequence). It is the only motion input any DPE algorithm actually integrates.
Is either unused?	<code>motion_hint</code> (singular) is effectively unused . It is attached to <code>TemporalRecord.motion_hint</code> for association only; its <code>angular_velocity_rad_s</code> is never read by any algorithm.
Angular-velocity units	radians per second, <code>(3,)</code> ndarray, about <code>frame_id</code> 's X/Y/Z axes
Coordinate frame	Declared per-sample via <code>MotionHint.frame_id</code> ; expected <code>FrameId.BODY</code> . Never inferred, never silently converted.
Timestamp units	Same opaque float convention as observation timestamps
Integration behaviour	<code>compute_rotation_compensation()</code> selects admissible samples spanning <code>[previous_timestamp, current_timestamp]</code> and integrates a short-window relative rotation
dt handling	Derived from <code>previous_for_comparison.timestamp</code> → <code>result_timestamp</code> ; <code>None</code> on a first/rejected/gap-restart frame, which is the explicit "no comparison interval" signal
Rotation compensation	Substitutes the previous record with a rotation-compensated copy before consistency/stabilization; never modifies this frame's own geometry
Temporal-admission relationship	Runs only on admitted frames (<code>ACCEPTED</code> / <code>ACCEPTED_NEW_SEQUENCE</code>)
Behaviour without motion	Fully legal. <code>rotation_compensation_status</code> = <code>NOT_APPLIED</code> ; no stage is blocked
Behaviour with malformed motion	Never raises. Degrades to <code>NOT_APPLIED</code> / <code>INSUFFICIENT_EVIDENCE</code>

Benchmark — identical stereo input, motion vs no motion (n=60 after 10 warmup)

Arm	median	mean	P95
A — no motion (<code>motion_hints=None</code>)	29.058 ms	29.223 ms	31.171 ms
B — valid representative motion (5-sample window, $\omega_z = 0.05$ rad/s)	31.317 ms	31.027 ms	32.658 ms
Δ (motion overhead)	+2.259 ms	—	—

Output differences

<code>depth_map_identical</code>	: True	<- motion NEVER alters raster geometry
<code>disparity_map_identical</code>	: True	
<code>rotation_compensation_status</code>	: NOT_APPLIED	-> APPLIED
<code>temporal_consistency_state</code>	: CONTRADICTIONARY	-> CONTRADICTIONARY
<code>motion_aware_reliability_state</code>	: UNRELIABLE	-> UNRELIABLE
<code>quality_overall_state</code>	: DEGRADED	-> DEGRADED

Motion changes only temporal *evidence* fields; every raster and 3-D geometry output is byte-identical. The ~2.3 ms overhead is the rotation-compensation stage plus motion-sample selection inside motion-aware reliability and persistence.

Malformed motion (never raises)

Input	Median	<code>rotation_compensation_status</code>	Motion reliability
NaN angular velocity	31.503 ms	APPLIED	INSUFFICIENT_EVIDENCE
<code>valid=False</code> flag	29.372 ms	NOT_APPLIED	UNRELIABLE
Empty sequence <code>[]</code>	28.554 ms	NOT_APPLIED	UNRELIABLE

Input	Median	rotation_compensation_status	Motion reliability
Stale window (100 s old)	28.300 ms	NOT_APPLIED	UNRELIABLE

Observation (P2): NaN angular velocity reports `rotation_compensation_status = APPLIED` while emitting `RuntimeWarning: invalid value encountered in cast` from `temporal/rotation_compensation.py:223-224`. The frame is not corrupted — `motion_aware_reliability` correctly falls to `INSUFFICIENT_EVIDENCE`, which is the field a consumer is meant to gate on, and `tests/test_d11_degradation_validation.py` already asserts no geometry is fabricated. But the `APPLIED` status is misleading in isolation. Pre-existing behaviour; not changed by D1.

14. Calibration / physical-unit contract

The authoritative contract

Boundary	Unit
StereoCalibration.Q translation term	MILLIMETRES
Focal length (Q[2,3])	pixels
Principal point (-Q[0,3], -Q[1,3])	pixels
Disparity	pixels; invalid convention is <code>disparity <= 0</code>
depth_map	metres (float32); invalid is exactly 0.0
PointCloud.points	metres; invalid is NaN
ObstacleCloud.distances_m	metres
FreeSpaceRays.ranges_m	metres
ClearanceEvidence.nearest_distance_m	metres; bearings in radians
RigidTransform.translation (body_T_camera_left)	metres
Depth clamp	[0.15 m, 8.0 m] — outside this range pixels are zeroed as invalid

Evidence. `depth/depth_estimator.py:132-133`:

```
# Calibration object-points were in mm → Z comes out in mm; convert to m.
depth = (depth_mm / 1000.0).astype(np.float32)
```

Corroborated by the shipped hardware calibration: `Q[3,2] = 0.0154497`, so `|1/Q[3,2]| = 64.726`. The repository documents this rig as a **64 mm** baseline (`pipeline_config.py`, `depth_estimator.py:29`). 64.726 is millimetres, not metres. `DepthEstimator.__init__` makes the same assumption explicitly: `self._baseline_m = abs(1.0 / tx) / 1000.0`.

Closed-form round trip through DPE's own `DepthEstimator`, mm-convention disparity in, metres out:

Target	Disparity (px)	DPE depth	Error
0.5 m	79.551	0.500 m	0.000
1.0 m	39.776	1.000 m	0.000
2.0 m	19.888	2.000 m	0.000
4.0 m	9.944	4.000 m	0.000
8.0 m	4.972	8.000 m	0.000

The Gazebo `baseline = 120.0` question — answered

`baseline = 120.0` for a physical 0.12 m stereo rig is CORRECT for DPE. No conversion is required.

DPE expects millimetres at the calibration boundary, and 120.0 mm = 0.12 m. Both candidates were built as real `Q` matrices and run through DPE's own `DepthEstimator` against a true 2.0 m plane:

Calibration writes	Q[3,2]	DPE reports for a true 2.0 m plane	Verdict
120.0 (millimetre convention)	0.008333	1.99999976 m	CORRECT
0.12 (metre convention)	8.333333	0.0 m — entire depth map invalid	WRONG

The dangerous direction is the opposite of the one that might be assumed. If a future Gazebo integration "corrects" `120.0` to `0.12` to match the SDF/URDF's metres, every depth becomes 1000× too small (2.0 m → 0.002 m), falls below `MIN_DEPTH_M = 0.15`, and is zeroed. The result is a **totally blank depth map**, not a subtly wrong one.

Required conversion, stated for later: if a simulation calibration producer natively emits the baseline in metres, the conversion at the DPE boundary is

```
Q[3, 2] = -1 / (baseline_metres * 1000.0)      # i.e. baseline_mm = baseline_m * 1000
P2[0, 3] = -focal_px * baseline_metres * 1000.0
```

No DPE or simulation source was modified. This is a boundary-adaptor concern for the future Gazebo integration, classified **P2** (Gazebo/hardware qualification), because the failure mode is loud (all-zero depth) rather than silent.

15. Main synchronous benchmark

Method. `DepthPerceptionPipeline.process_geometry_frame(StereoObservation) -> GeometryFrame` — the authoritative embedded path, **not** a legacy helper. **One** constructed pipeline instance per trial, reused across all frames. Construction excluded. 30 warmup frames discarded per trial, then 300 measured; **5 independent trials = 1500 samples**.

Recorded context: 320 × 240 · real hardware calibration · `rectify=True` · full V1 candidate geometry settings · full temporal stack enabled · 5-sample motion window per frame ($\omega_z = 0.05$ rad/s, $dt = 0.1$ s) · OpenCV 5.0.0 with 8 threads · load average 1.02 → 2.44.

Pooled distribution (n = 1500)

Statistic	Value
mean	30.406 ms
median	30.938 ms
P95	33.066 ms
P99	35.843 ms
min	27.163 ms
max	43.030 ms
stddev	1.971 ms
FPS (from mean)	32.89
FPS (from median)	32.32

Per-trial medians and the contention-robust estimator

```
per-trial medians (ms) : 28.29, 28.60, 31.41, 31.64, 31.52    (spread 3.35 ms)
best trial median      : 28.29 ms -> 35.35 FPS
p10 across all samples : 28.06 ms -> 35.63 FPS
minimum sample        : 27.16 ms
```

Authoritative figure: DPE median ≈ 30.9 ms (≈ 32 FPS) under realistic desktop load, with an uncontended floor of ≈ 28.3 ms (≈ 35 FPS). The 3.35 ms spread between trial medians is external load, not DPE variance — pooled stddev is under 2 ms and the max is only 1.4× the median.

Supplementary comparisons

Variant	median	FPS
<code>rectify=True</code> (authoritative)	30.94 ms	32.3
<code>rectify=False</code>	32.30 ms*	31.0
Legacy <code>process()</code> entry point	31.13 ms	32.1

* `rectify=False` is not faster, despite skipping `cv2.remap` entirely. Rectification itself costs only **0.628 ms** (measured directly in section 16), and skipping it here *adds* ~1.4 ms overall because of data-dependence: the synthetic fixture is constructed in already-rectified space, so leaving it unrectified preserves far more valid disparity (valid fraction **0.598 vs 0.231**) and every downstream geometry stage then processes ~3× more points (**45,945 vs 14,273** obstacle points). The lesson for HPE is that **DPE latency is driven by how much valid geometry a scene yields, not by fixed per-frame overhead** — a rich scene costs more than a sparse one.

The legacy `process()` figure (31.13 ms vs 30.94 ms) confirms the dual-interface layer adds no measurable overhead — `process_geometry_frame()` is not a slower path.

16. Stage-by-stage benchmark

Two independent, cross-checked views. **(A) In-situ:** the pipeline's own already-shipped `logger.debug("<Stage> stage: %.2f ms")` instrumentation, captured with a logging handler — *existing instrumentation, preferred as instructed; nothing added to production code*. **(B) Re-driven:** for the stages `pipeline.py` does not itself log, the same already-shipped functions called with the exact arguments `pipeline.py` passes them.

Total median 32.084 ms; **29.218 ms (91 %) accounted for**. Only stages that actually execute are listed.

Rank	Stage	Median	% of total	Source	Compute character
1	SGBM (<code>cv2.StereoSGBM.compute</code>)	6.064 ms	18.9 %	re-driven	native-heavy, multi-threaded
2	Point cloud (<code>PointCloudBuilder.build</code>)	3.930 ms	12.2 %	in-situ	NumPy, single-threaded
3	Free-space rays	2.910 ms	9.1 %	in-situ	NumPy
4	Obstacle cloud	2.790 ms	8.7 %	in-situ	NumPy
5	Scene interpretation (regions + decision)	2.767 ms	8.6 %	re-driven	NumPy + Python loop
6	Shadow-zone reliability mask	1.939 ms	6.0 %	re-driven	NumPy
7	Ramp-zone reliability mask	1.938 ms	6.0 %	re-driven	NumPy
8	Threat assessment (per-beam clearance)	1.517 ms	4.7 %	re-driven	NumPy + Python loop
9	Surface geometry	1.370 ms	4.3 %	in-situ	NumPy + per-cell Python loop
10	Body-frame transform	1.090 ms	3.4 %	in-situ	NumPy matmul
11	Rectification (<code>cv2.remap</code> ×2)	0.628 ms	2.0 %	re-driven	native, single-threaded at this size
12	Boundary geometry	0.590 ms	1.8 %	in-situ	NumPy + per-cell Python loop
13	Opening geometry	0.500 ms	1.6 %	in-situ	NumPy + Python loop
14	Rotation compensation	0.400 ms	1.2 %	in-situ	NumPy
15	Depth (Q reprojection, Z-only)	0.333 ms	1.0 %	re-driven	NumPy
16	Geometry metrics	0.100 ms	0.3 %	in-situ	NumPy
17	Motion-aware reliability	0.070 ms	0.2 %	in-situ	Python scalar
18	GeometryFrame assembly	0.067 ms	0.2 %	re-driven	Python
19	Temporal persistence	0.060 ms	0.2 %	in-situ	NumPy
20	Temporal stabilization	0.060 ms	0.2 %	in-situ	NumPy
21	Temporal consistency	0.050 ms	0.2 %	in-situ	NumPy
22	Grayscale (<code>cv2.cvtColor</code>)	0.026 ms	0.1 %	re-driven	native
23	Temporal admission	0.020 ms	0.1 %	in-situ	Python

- **Dominant stage: SGBM — 6.064 ms, 18.9 %.**
- **Second-largest: point-cloud construction — 3.930 ms, 12.2 %.**
- The remaining ~9 % is `build_result` assembly, region/clearance evidence extraction, and per-call allocation not individually instrumented.

The critical structural observation: SGBM — the one genuinely native, multi-threaded stage — is only **19 %** of DPE's frame time. **Roughly 81 % of DPE is single-threaded NumPy and Python.**

17. Temporal vs non-temporal benchmark

Both arms run the **same authoritative `process_geometry_frame()` path**; only supported configuration flags differ. The stateless/Tier-2 path was **not** substituted. Same observations, same instance discipline (n = 200 after 20 warmup).

Arm	median	mean	P95
A — full temporal stack	31.498 ms	31.549 ms	32.700 ms
B — temporal disabled via config	30.889 ms	31.116 ms	32.639 ms
Δ	+0.609 ms	+0.432 ms	+0.060 ms

The entire Level-4 temporal stack — admission, rotation compensation, consistency, stabilization, motion-aware reliability and persistence — costs 0.61 ms, or 1.9 % of frame time. The P95 delta (+0.06 ms) is within run-to-run noise, so temporal processing does not measurably worsen tail latency either.

Output differences

```
raster geometry (depth_map) : BYTE-IDENTICAL between arms  <- temporal is strictly additive
depth_valid_fraction       : 0.2502 == 0.2502

A: consistency=CONTRADICTIONY  stabilization=STABILIZED  rotation=APPLIED
   reliability=UNRELIABLE      persistence=UNRELIABLE
   degradation_reasons = [GEOMETRY_VALIDITY:DEGRADED, TEMPORAL_CONSISTENCY:DEGRADED,
                           MOTION_RELIABILITY:DEGRADED, PERSISTENCE:DEGRADED]

B: consistency=None  stabilization=None  rotation=None
   reliability=None  persistence=None
   degradation_reasons = [GEOMETRY_VALIDITY:DEGRADED]
```

This confirms the documented raw-vs-temporal authority rule empirically: temporal evidence never rewrites Level-3 geometry. It also demonstrates the absent-vs-degraded distinction — arm B reports `None` (capability unavailable), not a false healthy state.

(The `CONTRADICTIONY` / `UNRELIABLE` states reflect the benchmark scene deliberately changing texture every frame, which exercises the full temporal path rather than a static-scene shortcut.)

18. Resolution scaling

Resolution	Calibration	Total median	FPS	SGBM	Rectification	Geometry + other
320 × 240	hardware fixture — QUALIFIED	31.51 ms	31.7	5.94 ms	0.65 ms	24.92 ms
640 × 480	exactly-derived 2× scaling — NOT a qualified configuration	172.33 ms	5.8	21.50 ms	1.07 ms	149.75 ms

The 640 × 480 calibration is derived, not fabricated, and is clearly labelled as non-qualified. For an ideal pinhole camera, sampling the same sensor at 2× pixel density scales `fx`, `fy`, `cx`, `cy` by exactly 2 and leaves the normalized distortion coefficients and the physical baseline unchanged. `Q[3,2] = -1/Tx` was left **exactly untouched** — it is a pure function of the physical baseline — so metric depth is preserved rather than silently rescaled. No invalid calibration was invented.

Scaling factors (4× the pixels)

Component	Ratio	Interpretation
Pixels	4.00×	reference
SGBM	3.62×	sub-linear — benefits from its native 8-thread parallelism
Rectification	1.64×	strongly sub-linear — <code>cv2.remap</code> amortizes fixed overhead
Geometry + other stages	6.01×	super-linear
Total latency	5.47×	
FPS	31.7 → 5.8	

Geometry stages scale far worse than SGBM. At 4× the pixels the single-threaded NumPy geometry stages cost **6.0×** more while multi-threaded SGBM costs only **3.6×** more — SGBM actually scales *better* than pixel count because its native parallelism has more work to spread across 8 threads, while the geometry stages get no such benefit *and* process disproportionately more points as higher resolution recovers more valid disparity (valid fraction 0.244 → 0.309).

At 320×240 SGBM is 19 % of the frame; at 640×480 it falls to 12 %, while geometry rises to 87 %. **Raising DPE resolution is dominated by the single-threaded NumPy geometry stages, not by SGBM.** This is the single most important scaling fact for any future HPE resolution decision, and it points at the geometry stages — not the stereo matcher — as the optimization target if DPE latency ever needs to come down.

19. Repeated-run stability

400 sequential frames through the SAME `DepthPerceptionPipeline` instance (qualified config, motion window per frame).

Check	Result
Crash / exception	None
Latency median	31.42 ms (P95 33.76 ms)
Quarter medians (ms)	31.76 → 31.33 → 31.42 → 31.29
Latency drift (Q4 vs Q1)	−0.47 ms = −1.48 % — <i>decreasing</i> ; no progressive explosion
RSS before / after / after GC	236.2 / 178.4 / 178.4 MB
RSS growth over 400 frames	−57.8 MB (RSS fell, then stayed flat)
Temporal history bounded	Yes — pinned at exactly 30 / <code>temporal_max_records</code> = 30 from frame 50 onward
Persistence state bounded	Grid fixed at (60, 80) from frame 0; never grows with frame count
EMA/debounce bounded	<code>_ema_dist</code> length constant at 20 (= <code>n_beams</code>)
Output accumulation	None — <code>boundary_evidence</code> = 12, <code>opening_evidence</code> = 2, obstacle points 14.9k-16.2k, all oscillating with scene content, never growing
Identity leakage	None observable (there is no observation identity to leak — see section 6)
Major object reconstruction	None — all 8 watched objects retained identical <code>id()</code> across all 400 frames
Matcher / rectifier identity	Stable

The negative RSS figure is an artefact of the preceding section, not of DPE: the stability run started while the interpreter still held the 640×480 buffers allocated by the resolution-scaling benchmark. Those were reclaimed early, after which **RSS was exactly flat at 178.4 MB for the remaining 350 frames** (frames 50, 100, 150, 200, 250, 300, 350, 399 all read 178.4 MB). History was clamped at 30 from frame 50 onward, the persistence grid stayed fixed at (60, 80), and per-frame output sizes oscillated with scene content without trending. **DPE is safe for indefinite long-running embedded use.**

20. Frame-order / drop / discontinuity behavior — CRITICAL for HPE backpressure

What temporal history actually keys on

`TemporalRecord.timestamp` — and nothing else.

`temporal/history.py::admit()` compares only `record.timestamp` against `self._records[-1].timestamp`. There is **no sequence number**, **no arrival counter**, and **no wall-clock read** — the module never calls `time.time()` or `time.perf_counter()`, which is why it behaves identically for simulated, recorded and live timestamps. A repository-wide search confirms no sequence-number concept exists anywhere in DPE.

Arrival order matters only in that `admit()` compares against the *newest admitted* record, so an out-of-order submission is **rejected**, never re-sorted. History is never reordered or mutated.

Measured: skipped observations (the 100 / 101 / 105 case)

Submitting only observations 100, 101, 105, 106, 110 — intermediate captures never handed to DPE:

submitted ts	history	consistency	stabilization	reliability	persistence	quality
0.0	1	INSUFFICIENT_EVIDENCE	INSUFFICIENT_EVIDENCE	INSUFFICIENT_EVIDENCE	CLASSIFIED	DEGRADED
0.1	2	CONTRADICTIONARY	STABILIZED	UNRELIABLE	UNRELIABLE	DEGRADED
0.5	3	CONTRADICTIONARY	STABILIZED	UNRELIABLE	UNRELIABLE	DEGRADED
0.6	4	CONTRADICTIONARY	STABILIZED	UNRELIABLE	UNRELIABLE	DEGRADED
1.0	5	CONTRADICTIONARY	STABILIZED	UNRELIABLE	UNRELIABLE	DEGRADED

Identical state progression to the fully contiguous 10-frame run. History grows 1→2→3→4→5 exactly as it does with no drops. Nothing rejects, resets, or degrades because captures were skipped.

Other discontinuity characterizations

Scenario	Behaviour
Duplicate timestamp	<code>REJECTED_DUPLICATE_TIMESTAMP</code> . History unchanged (2 → 2). Geometry still returned in full. <code>temporal_consistency/stabilization/rotation</code> → <code>None</code> ; reliability → <code>INSUFFICIENT_EVIDENCE</code> . Recovers cleanly on the next valid timestamp.
Decreasing timestamp	<code>REJECTED_OLDER_TIMESTAMP</code> . Same shape as duplicate. History never reordered.
Forward jump 1.0 s (< gap_limit_s=5.0)	Absorbed. <code>ACCEPTED</code> , history keeps growing 2→3→4, temporal states continue normally.
Forward jump 30 s (> gap_limit_s)	<code>ACCEPTED_NEW_SEQUENCE</code> . History cleared to 1 , persistence tracker cleared , temporal states correctly reset to <code>INSUFFICIENT_EVIDENCE</code> . Self-healing — no external <code>reset()</code> needed.

Does DPE require every captured frame, or every submitted frame?

Only every submitted frame. DPE has no concept of a capture it never saw.

Classification

A — the algorithm remains valid, but temporal confidence/history reflect the larger gap.

Not B: DPE does **not** silently treat non-consecutive observations as consecutive — the actual timestamp delta is what every temporal threshold is measured against, so a wider gap is genuinely seen as a wider gap. Not C: DPE does not reject discontinuity; it either absorbs it or explicitly restarts the sequence with a reported status. Not D.

May HPE drop an observation BEFORE DPE begins processing it?

YES — safely. A never-submitted observation is invisible to DPE. Temporal history keys purely on timestamp, so the only effect of dropping is a larger inter-frame `dt`, which every temporal stage already measures correctly. If the resulting gap exceeds `temporal_gap_limit_s`, DPE self-restarts the sequence and reports `ACCEPTED_NEW_SEQUENCE` rather than producing silently-degraded evidence.

One caveat HPE must respect: dropping is safe, but **reordering is not**. An observation submitted out of order is rejected from temporal history (though it still returns a full, valid `GeometryFrame` — only the temporal evidence fields go `None`). See Q5 in section 27.

Second caveat (new finding): motion-aware reliability and the persistence tracker run **outside** the "was this frame admitted?" block (`pipeline.py:862, 902`). A chronology-**rejected** frame therefore still mutates persistence tracker state. This is deliberate (a rejected frame must still be classified `INSUFFICIENT_EVIDENCE` rather than silently skipped) but it means "rejected" $\neq$ "no side effect". Classified **P3 — informational**. No policy implemented.

21. Failure semantics

21. Failure semantics

Case	Classification	Behaviour	Raises	Returns GeometryFrame	Quality / degradation set
Well-formed baseline	<i>(control)</i>	normal	—	✓	DEGRADED, valid frac 0.256
Mismatched L/R dimensions	CALLER CONTRACT VIOLATION	<code>ValueError: left_image shape (240,320) does not match right_image shape (120,160).</code>	✓	✗	—
Frame size $\neq$ calibration	CALLER CONTRACT VIOLATION	<code>ValueError: left frame size (160×120) does not match calibrated size (320×240).</code>	✓	✗	—
<code>left_image = None</code>	CALLER CONTRACT VIOLATION	<code>ValueError: left_image is None.</code>	✓	✗	—
float64 dtype pair	CALLER CONTRACT VIOLATION	<code>cv2.error</code> (raw OpenCV, from <code>cvtColor</code>)	✓	✗	—
Grayscale pair	<i>(supported input)</i>	normal	—	✓	DEGRADED
Channel-count mismatch (BGR left, gray right)	CALLER CONTRACT VIOLATION (silent)	Accepted; output identical to baseline	✗	✓	DEGRADED
Textureless pair (no disparity)	RECOVERABLE FRAME DEGRADATION	valid frac 0.000	✗	✓	INSUFFICIENT
Decorrelated pair (no valid depth)	RECOVERABLE FRAME DEGRADATION	valid frac 0.008	✗	✓	INSUFFICIENT
Missing motion	<i>(legal, unremarkable)</i>	normal	✗	✓	DEGRADED
Malformed motion (NaN)	TEMPORAL DEGRADATION	<code>RuntimeWarning</code> emitted; no crash	✗	✓	DEGRADED; reliability <code>INSUFFICIENT_EVIDENCE</code>
No timestamp	TEMPORAL DEGRADATION	<code>REJECTED_INVALID_TIMESTAMP</code>	✗	✓	DEGRADED
NaN timestamp	TEMPORAL DEGRADATION	<code>REJECTED_INVALID_TIMESTAMP</code>	✗	✓	DEGRADED
Temporal admission rejection (older ts)	TEMPORAL DEGRADATION	temporal fields $\rightarrow$ <code>None</code>	✗	✓	DEGRADED
Image matches a <i>different</i> calibration	CALLER CONTRACT VIOLATION	<code>ValueError: left frame size (640×480) does not match calibrated size (320×240).</code>	✓	✗	—
<code>rectify=False</code> + oversized image	CALLER CONTRACT VIOLATION (silent)	Accepted; returns plausible geometry, valid frac 0.799 — computed against the wrong calibration	✗	✓	INSUFFICIENT
Call after <code>close()</code>	FATAL ENGINE ERROR	<code>RuntimeError: ... called after close() – construct a new pipeline instead of reusing a closed one.</code>	✓	✗	—

How HPE distinguishes a provider exception from a valid degraded frame. The separation is clean and usable:

- **An exception means the frame is untrustworthy and must be dropped.** DPE deliberately wraps *no* stage in `try/except` — `pipeline.py`'s own comments explain that a rectification or geometry failure must invalidate the whole frame rather than silently degrade, because a caller has no way to trust depth computed from unrectified images against calibration-derived maps.
- **A returned `GeometryFrame` is always structurally valid**, even when the scene yielded nothing. Degradation is reported *in-band* via `quality.overall_state` (`VALID` / `DEGRADED` / `INSUFFICIENT`) and `quality.degradation_reasons` (`"{DIMENSION}:{STATE}"`), never by throwing.

Two caveats for HPE's exception handling:

1. **DPE does not raise a uniform exception type.** A float64 image pair escapes as a raw `cv2.error`, not a DPE `ValueError`. **HPE must catch `Exception`, not `(ValueError, RuntimeError)`**. Classified **P1**.
 2. **Two contract violations are accepted silently.** A channel-count mismatch is benign in practice, but `rectify=False` with a wrongly-sized image returns confident, plausible-looking geometry derived from the wrong calibration, with no signal at all. Classified **P2** — it requires `rectify=False`, which an HPE integration has no reason to use.
-

22. Native / Python compute characterization

Method (no concurrency introduced). Each stage was re-timed under `cv2.setNumThreads(1)` and `cv2.setNumThreads(8)` in the same process, **interleaved A/B/A/B per stage** (3 repeats per arm, median of repeats), with both arms warmed before either was timed. Interleaving matters: an initial measure-all-of-A-then-all-of-B pass produced an apparent 1.05–1.98× "speedup" on every stage, including `body transform`, which contains no OpenCV call at all — that was warm-cache/allocator bias, and it disappears entirely under interleaving. A stage that still speeds up when OpenCV is given more threads is executing inside OpenCV's own native parallel region, which by construction runs outside the Python interpreter.

Stage	1 thread	8 threads	Speedup	Classification
SGBM (<code>cv2.StereoSGBM.compute</code>)	11.138 ms	5.753 ms	1.94×	NATIVE-HEAVY — multi-threaded
Rectification (<code>cv2.remap</code>)	0.633 ms	0.632 ms	1.00×	native, single-threaded at this size
Grayscale (<code>cv2.cvtColor</code>)	0.026 ms	0.026 ms	1.00×	native, single-threaded
Depth (NumPy Q reprojection)	0.317 ms	0.318 ms	1.00×	PYTHON/NumPy-bound
Point cloud (NumPy)	3.784 ms	3.762 ms	1.01×	PYTHON/NumPy-bound
Body transform (NumPy matmul)	0.893 ms	0.905 ms	0.99×	PYTHON/NumPy-bound
Obstacle cloud (NumPy)	2.516 ms	2.491 ms	1.01×	PYTHON/NumPy-bound
Shadow-zone mask (NumPy)	1.928 ms	1.942 ms	0.99×	PYTHON/NumPy-bound
Ramp-zone mask (NumPy)	2.084 ms	2.077 ms	1.00×	PYTHON/NumPy-bound
Surface evidence	1.227 ms	1.228 ms	1.00×	MIXED (NumPy + per-cell Python loop)
Boundary evidence	0.403 ms	0.400 ms	1.01×	MIXED (NumPy + per-cell Python loop)
Scene interpretation	2.815 ms	2.825 ms	1.00×	MIXED
Threat assessment	1.612 ms	1.605 ms	1.00×	MIXED

The result is unambiguous: SGBM at 1.94× is the only stage that responds to OpenCV threading. Every other stage sits in 0.99–1.01×, i.e. no effect whatsoever.

One nuance worth recording: `geometry/rigid_transform.py:97` performs `flat_points @ transform.rotation.T`, a NumPy matmul that dispatches to **scipy-openblas 0.3.33** (`DYNAMIC_ARCH`, `MAX_THREADS=64`), and no `OMP_NUM_THREADS` / `OPENBLAS_NUM_THREADS` cap is set in this environment. That stage is therefore *potentially* natively threaded by a runtime independent of OpenCV — but the (3, 3) rotation matrix is far too small for OpenBLAS to thread, and the measurement confirms no effect (0.99×). Recorded because a future HPE that sets thread-pool environment variables would be affecting **two** independent native runtimes inside DPE, not one.

Python-level loop counts in production modules (AST scan): `boundary.py` 5, `opening.py` 5, `rotation_compensation.py` 3, `reliability.py` 2, `surface.py` 2, `scene_interpreter.py` 2, `history.py` 2, plus single loops elsewhere. All are **per-grid-cell or per-beam** (grids are 3×3 to 3×6, beams 20) — never per-pixel. `tests/test_performance_guards.py` structurally enforces zero `for/while` loops in the five fully-vectorized geometry modules.

Note: `cv2.reprojectImageTo3D` is **not** used. `DepthEstimator.estimate()` deliberately computes the Z channel directly in NumPy.

GIL statement — carefully bounded

Only SGBM demonstrably executes outside the interpreter. Its 1.94× speedup from OpenCV threading is direct evidence of a native parallel region; OpenCV releases the GIL around such regions. That is **6.1 ms of a ~32 ms frame — about 19 %**.

For the NumPy stages, no GIL-release claim is made. NumPy releases the GIL around some large vectorized kernels, but the arrays here are small (240×320) and the observed absence of any threading speedup provides no evidence either way. **This report does not claim DPE releases the GIL for ~81 % of its work — the evidence supports the opposite reading: most of DPE is single-threaded, likely GIL-holding Python/NumPy.**

Is HPE-level threading between DPE and NPE plausible?

Yes, and for a better reason than GIL release. NPE runs on **ONNX Runtime CPU**, which executes its entire inference in native code with the GIL released and its own internal thread pool. DPE spends ~81 % of its time single-threaded. So a DPE thread and an NPE thread would contend for **cores**, not primarily for the interpreter lock — and DPE only genuinely wants more than one core during its 6 ms SGBM window. See section 25.

23. DPE vs frozen NPE

	DPE (measured this run)	NPE v2.0.0 (frozen, 2dac0799...)
Median latency	30.938 ms (pooled) / 28.290 ms (uncontended)	95.3 ms
Standalone FPS	32.32 / 35.35	10.49
Breakdown	SGBM 6.06 ms + geometry ~26 ms	YOLOX 10.1 ms + RoadSeg 78.0 ms
Runtime	OpenCV 5.0.0 + NumPy, CPU	ONNX Runtime, CPU EP
Dominant cost	single-threaded NumPy geometry (~81 %)	RoadSeg inference (82 %)

DPE is $\approx 3.1\times$ faster than NPE. NPE is decisively the system bottleneck: at 95.3 ms it alone caps any pipeline containing it to 10.5 FPS.

24. Theoretical provider-overlap opportunity

Using the **uncontended** DPE median of **28.290 ms** (the conservative choice — it makes the overlap opportunity look *smaller*, not larger):

```
sequential_provider_wall      = DPE_median + NPE_median
                              = 28.290 + 95.3          = 123.590 ms   ->   8.09 FPS

theoretical_independent_floor = max(DPE_median, NPE_median)
                              = max(28.290, 95.3)     = 95.300 ms   ->  10.49 FPS

theoretical_maximum_overlap_saving
                              = 123.590 - 95.300      = 28.290 ms   ->  22.9 %
```

Using the pooled median of 30.938 ms instead: sequential 126.24 ms (7.92 FPS), floor 95.3 ms (10.49 FPS), saving 30.94 ms (24.5 %).

Schedule	Wall time	FPS	vs sequential
Sequential DPE → NPE	123.59 ms	8.09	—
Perfect overlap (theoretical floor)	95.30 ms	10.49	+29.7 % throughput
Maximum possible saving	28.29 ms		22.9 % of wall

These are arithmetic upper bounds only. They assume zero contention, zero join cost, zero queueing, and zero scheduling overhead. **No concurrent execution was measured in D1, and no claim of real concurrent performance is made.** The realistic saving will be lower — see section 25.

The strategic conclusion is that overlap is worth *at most* ~23 %, and DPE is not the thing to optimize. Even reducing DPE to zero would only take the system from 8.09 FPS to 10.49 FPS. NPE's 95.3 ms — and RoadSeg's 78 ms within it — is the real ceiling.

25. CPU-contention risks

Machine: 4 physical cores / 8 hardware threads. OpenCV configured for 8 threads. ONNX Runtime defaults to its own thread pool, typically also sized to core count.

Risk assessment:

1. **Thread oversubscription is the primary risk.** OpenCV's SGBM wants 8 threads and ONNX Runtime independently wants ~4-8. Run concurrently with default settings, ~16 runnable threads would compete for 4 physical cores. The measured pinning experiment already demonstrates this class of harm: restricting OpenCV to 4 CPUs made DPE **worse** (40.8 ms vs 37.5 ms). Note also that DPE contains **two** independent native runtimes that read thread-pool settings — OpenCV and, via NumPy matmul, scipy-openblas — so a co-allocation scheme must account for both.
2. **The exposure window is narrow, which is good news.** DPE only genuinely wants multiple cores during SGBM — **6.1 ms of a 32 ms frame (19 %)**. For the other ~81 % DPE is single-threaded and occupies exactly one core, leaving the remaining three free for ONNX Runtime. The DPE/NPE contention window is therefore small and bounded.
3. **Memory bandwidth, not just cores.** DPE's NumPy geometry stages are memory-bandwidth-bound (repeated full-raster passes: point cloud, body transform, two reliability masks, obstacle extraction). RoadSeg inference is also bandwidth-hungry. On a shared LPDDR4x laptop memory controller they will contend even when core counts look adequate. This is not captured by any core-count model.
4. **Thermal and frequency headroom.** The CPU governor is `powersave`, base 2.40 GHz, turbo 4.20 GHz. Sustained dual-provider load will reduce achievable turbo, so both providers will be individually slower under concurrency than their measured standalone numbers.
5. **Measurement realism.** All DPE numbers here were taken on a contended developer laptop with only ~0.7-1.2 GiB free RAM. Production numbers on a dedicated target may differ materially in either direction.

Recommendation for whoever implements HPE concurrency (not implemented here): the thread budgets of OpenCV and ONNX Runtime must be explicitly co-allocated (e.g. `cv2.setNumThreads(N)` + ONNX Runtime `intra_op_num_threads`) rather than both defaulting to the full core count. Given DPE's 19 % multi-threaded window and NPE's dominance, biasing cores toward NPE is likely optimal. **This should be measured, not assumed.**

26. Exact HPE-facing DPE contract

The proposed frozen integration statement, derived **only** from measured current DPE behaviour.

IMPORT BOUNDARY

```
from depth_perception_engine.core import (
    DepthPerceptionPipeline, PipelineConfig, StereoCalibration,
    StereoObservation, MotionHint, GeometryFrame, FrameId, RigidTransform,
)

HPE imports NOTHING from depth_perception_engine.standalone, and
nothing from depth_perception_engine.pipeline.api. Importing `core`
provably does not load the standalone layer.
```

CONSTRUCTION

ONCE, at HPE startup

```
pipeline = DepthPerceptionPipeline(
    config,                # PipelineConfig
    calibration,           # StereoCalibration, owned by HPE
    rectify=True,
    body_T_camera_left=RigidTransform(..., CAMERA_OPTICAL_LEFT, BODY),
)

Cost: 0.157 ms. Never per-frame.
```

RUNTIME METHOD

EXACTLY ONE

```
geometry: GeometryFrame = pipeline.process_geometry_frame(observation)
```

INPUT TYPE depth_perception_engine.core.StereoObservation

OUTPUT TYPE depth_perception_engine.core.GeometryFrame (never None)

IMAGE OWNERSHIP

Images are held BY REFERENCE and never copied. HPE MUST NOT mutate an image after handing it to DPE and before the call returns. DPE returns freshly allocated output arrays every call; no output aliases an input.

CALIBRATION OWNERSHIP

HPE owns and supplies the StereoCalibration at CONSTRUCTION.
StereoObservation.calibration is RESERVED AND UNREAD – setting it has no effect. Changing calibration requires constructing a new pipeline;
reset() explicitly retains calibration and rectification maps.
Q's translation term MUST be in MILLIMETRES.

MOTION OWNERSHIP

HPE supplies observation.motion_hints – a bounded Sequence[MotionHint] spanning the interval leading to this capture, angular velocity in rad/s, frame declared per-sample (expected FrameId.BODY).
observation.motion_hint (singular) is association-only and is NOT read by any algorithm. Motion is always optional; absence degrades cleanly to rotation_compensation_status=NOT_APPLIED and never blocks perception.

TIMESTAMP BEHAVIOUR

left_timestamp wins over right_timestamp; the chosen value passes through verbatim to GeometryFrame.timestamp. Opaque float, no unit enforced – but SECONDS is required in practice whenever enable_temporal is True. Must be strictly increasing across submissions or the frame is excluded from temporal history (geometry is still returned).

COORDINATE-FRAME BEHAVIOUR

GeometryFrame.frame_id is ALWAYS the constant "camera_optical_left" and describes the coordinate system of the top-level rasters. Nested evidence declares its own frame: geometry_body / obstacle_cloud / free_space_rays / surface_evidence -> "body";
boundary / clearance / region evidence -> "camera_optical_left".

OBSERVATION-IDENTITY BEHAVIOUR

*** CONTRACT GAP ***

DPE carries NO observation transaction identity on its output.
StereoObservation.frame_id is never read and never propagates.
GeometryFrame exposes no immutable per-observation correlator.
The only correlator available is `timestamp`, which is optional,
caller-defined, and not guaranteed unique.

STATEFULNESS

STATEFUL. One instance owns per-beam EMA/debounce, a bounded temporal history ($\leq$ temporal_max_records), and a per-cell persistence tracker.
Frame-to-frame semantics genuinely depend on this state.

ORDERING REQUIREMENTS

Submissions MUST be non-decreasing in timestamp. Gaps are permitted and safe. Out-of-order or duplicate submissions are rejected from temporal history (temporal evidence fields become None) but STILL return a full, structurally valid GeometryFrame.

CONCURRENCY

ONE ACTIVE INVOCATION AT A TIME per instance. Not thread-safe; no internal synchronization exists.

RESET SEMANTICS

reset() -> complete temporal amnesia (EMA/debounce rebuilt, history cleared, persistence cleared, counters zeroed);
RETAINS calibration, config, rectifier, matcher.
close() -> terminal; every process*/reset call afterwards raises
RuntimeError. health() still readable. Idempotent.

FAILURE BEHAVIOUR

RAISES -> the frame is untrustworthy; drop it. Exception type is NOT uniform: expect ValueError, RuntimeError, AND raw cv2.error.
HPE must catch Exception.
RETURNS -> the frame is structurally valid. Degradation is reported IN-BAND via quality.overall_state (VALID/DEGRADED/INSUFFICIENT) and quality.degradation_reasons. A returned frame is never an error.

EXECUTION POLICY

DPE owns NO execution policy. No queue, no worker, no buffer, no backpressure, no expiry, no ordering machinery exists inside DPE, and none should be added. HPE owns all of it.

Contract gap, stated plainly: *current DPE cannot satisfy exact HPE observation correlation.* The [GeometryFrame](#) contract exposes no immutable observation identity, so an exact ID-based join of [GeometryFrame X](#) with [SemanticFrame X](#) is not expressible today. This must be resolved before HPE asynchronous orchestration can be correct. See section 28, P0-1.

27. Ten orchestration-readiness answers

Q1. Can DPE remain a synchronous provider? YES. `process_geometry_frame()` is a plain blocking call: `StereoObservation` in, `GeometryFrame` out, 30.9 ms median, no callbacks, no futures, no internal scheduling. It contains no execution policy of any kind, which is exactly what the HPE architecture requires of a provider.

Q2. Should HPE construct one DPE instance and reuse it? YES. Construction is 0.157 ms and builds the rectification maps, SGBM matcher, and all temporal stores. Over 400 frames all eight major objects retained identical `id()`. More importantly, correctness requires it: per-beam EMA/debounce, temporal history, and per-cell persistence only exist across frames on a reused instance. `pipeline.api.process_stereo_pair()` builds a fresh pipeline per call and therefore has *no* temporal capability at all.

Q3. Should only one DPE invocation be active at a time? YES. Every frame performs read-modify-write on `ThreatAssessor.ema_dist` / `pending_count`, `TemporalHistory.records`, and `TemporalPersistenceTracker.support_count`; the `cv2.StereoSGBM` matcher is shared and owns internal scratch buffers; and a repository-wide scan found **zero** synchronization primitives. Two concurrent calls would corrupt EMA smoothing, debounce counting, chronology ordering, and persistence support.

Q4. Is it architecturally plausible for HPE to run one DPE worker and one NPE worker independently? CONDITIONALLY YES. Architecturally the providers are cleanly separable — DPE is synchronous, stateful-per-instance, and owns no execution policy. Two conditions apply. (i) **Thread budgets must be co-allocated:** OpenCV wants 8 threads and ONNX Runtime independently wants ~4–8 on a 4-core machine; the measured pinning experiment shows how badly naive core restriction hurts. (ii) **The payoff is bounded at $\leq 22.9\%$** (section 24) and both providers will run individually slower under contention. Favourably, DPE only wants multiple cores during its 6.1 ms SGBM window (19 % of the frame), so the contention window is narrow.

Q5. Does DPE require submitted observations to remain in processing order? YES — for temporal evidence; NO — for geometry. `TemporalHistory.admit()` rejects any timestamp that is not strictly greater than the newest admitted one (`REJECTED_OLDER_TIMESTAMP` / `REJECTED_DUPLICATE_TIMESTAMP`) and never reorders history. An out-of-order submission still returns a complete, structurally valid `GeometryFrame` — only `temporal_consistency`, `temporal_stabilization` and `rotation_compensation_status` become `None`. **HPE must preserve non-decreasing timestamp order when submitting.**

Q6. Can HPE drop an observation BEFORE submitting it to DPE when overloaded? YES. Measured directly: submitting 100, 101, 105, 106, 110 produced an *identical* state progression to a fully contiguous run. A never-submitted observation is invisible to DPE — there is no sequence number, no arrival counter, and no wall-clock read anywhere in the temporal module.

Q7. Does dropping observations break DPE temporal semantics? NO — classification A. Temporal semantics remain valid; confidence and history simply reflect the larger real `dt`, which every temporal threshold already measures correctly. If the gap exceeds `temporal_gap_limit_s`, DPE self-restarts (`ACCEPTED_NEW_SEQUENCE`), clearing both history and persistence, and reports it — rather than silently treating distant frames as adjacent. Measured with a 30 s jump: history cleared to 1, states correctly reset to `INSUFFICIENT_EVIDENCE`.

Q8. Does DPE need an internal queue of its own? NO. DPE processes exactly one observation per blocking call and holds no pending work. Adding a queue would make DPE own execution policy, which the architecture explicitly reserves for HPE. No queue, `threading`, `asyncio`, `multiprocessing`, or `concurrent` import exists in production source today, and none should be added.

Q9. Does DPE need an internal result buffer of its own? NO. `process_geometry_frame()` returns its `GeometryFrame` directly to the caller and retains no reference to it. The only retained per-frame data is the bounded `TemporalHistory` (≤ 30 decimated depth snapshots, measured pinned at exactly 30) and the fixed-shape persistence grid — these are *algorithmic* state, not a result buffer, and RSS was flat at 178.4 MB across the steady-state portion of a 400-frame run. Result buffering, expiry, and output ordering belong to HPE.

Q10. What exact DPE public call should HPE invoke?

```
from depth_perception_engine.core import DepthPerceptionPipeline, StereoObservation, GeometryFrame

# once, at startup
pipeline = DepthPerceptionPipeline(config, calibration, rectify=True,
                                   body_T_camera_left=body_T_camera_left)

# per observation, one active call at a time, non-decreasing timestamps
geometry: GeometryFrame = pipeline.process_geometry_frame(
    StereoObservation(
        left_image=left, right_image=right,
        left_timestamp=t, motion_hints=hints,
    )
)
```

28. HPE blocker classification

P0 — must fix before HPE asynchronous orchestration can be correct

#	Issue	Evidence
P0-1	DPE carries no observation transaction identity on <code>GeometryFrame</code>. HPE's exact same-observation join of <code>GeometryFrame X</code> with <code>SemanticFrame X</code> is not expressible against the current public contract. <code>StereoObservation.frame_id</code> is declared but never read (zero production source references); the probe string " <code>observation-X</code> " appears nowhere in the returned object graph; <code>GeometryFrame.frame_id</code> is the hardcoded coordinate-frame constant <code>camera_optical_left</code> ; <code>timestamp</code> is optional, caller-defined and not guaranteed unique. DPE needs two distinct concepts — an immutable <code>observation_id</code> and the existing <code>coordinate_frame</code> — and <code>frame_id</code> must not be repurposed. Not fixed in D1.	Sections 5, 6, 7

P1 — should fix before HPE implementation, but not conceptually blocking

#	Issue	Evidence
P1-1	Non-uniform exception type. A float64 image pair escapes as a raw <code>cv2.error</code> , not a DPE-owned exception. HPE cannot safely catch (<code>ValueError</code> , <code>RuntimeError</code>) and must catch <code>Exception</code> , which weakens its ability to distinguish a provider bug from a caller contract violation.	Section 21
P1-2	Timestamp unit is undefined but load-bearing. Docstrings call it "opaque, caller-defined", yet <code>temporal_max_age_s</code> / <code>temporal_gap_limit_s</code> compare it directly as seconds. A caller supplying nanoseconds or milliseconds would silently get wrong gap/age behaviour with no error. Worth stating explicitly in the frozen contract.	Section 7
P1-3	<code>pipeline.temporal_history</code> leaks a Level-4 internal type through a public property. Harmless today, but it should not appear in an HPE-facing contract.	Section 4

P2 — can defer to Gazebo / hardware qualification

#	Issue	Evidence
P2-1	Gazebo calibration unit boundary. DPE requires <code>Q</code> in millimetres . <code>baseline = 120.0</code> for a 0.12 m rig is already correct and needs no conversion — but "correcting" it to <code>0.12</code> yields an entirely blank depth map. The boundary adapter must be explicit. Loud failure mode, hence P2.	Section 14
P2-2	<code>rectify=False</code> with a mismatched image size silently returns plausible-but-wrong geometry computed against the wrong calibration, with no signal. Requires <code>rectify=False</code> , which an HPE integration has no reason to use.	Section 21
P2-3	NaN angular velocity reports <code>rotation_compensation_status = APPLIED</code> while emitting <code>RuntimeWarnings</code> from <code>rotation_compensation.py:223-224</code> . Not corrupting (<code>motion_aware_reliability</code> correctly falls to <code>INSUFFICIENT_EVIDENCE</code> , and the existing test suite asserts no geometry is fabricated) but misleading in isolation. Pre-existing.	Section 13
P2-4	Resolution scaling is dominated by single-threaded geometry, not SGBM. 4× the pixels costs 5.47× the time, with geometry stages at 6.01× vs SGBM's 3.62×. At 640×480 geometry is 87 % of the frame. Any future resolution increase must be re-qualified.	Section 18

P3 — informational / performance only

#	Issue	Evidence
P3-1	Motion-aware reliability and the persistence tracker run outside the "was this frame admitted?" block, so a chronology-rejected frame still mutates persistence state. Deliberate, but "rejected" ≠ "no side effect".	Sections 3, 20
P3-2	A channel-count mismatch (BGR left + grayscale right) is silently accepted. Benign in practice — output was byte-identical to the well-formed baseline.	Section 21
P3-3	~81 % of DPE frame time is single-threaded NumPy/Python. The dominant single stage (SGBM, 18.9 %) is the <i>only</i> multi-core one (1.94× under OpenCV threading; every other stage measures 0.99-1.01×). If DPE latency ever needs to drop, the target is the geometry stages, not SGBM.	Sections 16, 22
P3-4	Theoretical provider-overlap saving is capped at 22.9 % . NPE at 95.3 ms is the system ceiling; DPE is not the thing to optimize.	Section 24
P3-5	DPE latency is scene-dependent, not fixed. A scene yielding 0.60 valid-depth fraction cost ~1.4 ms more than one yielding 0.23, because every geometry stage processes ~3× more points (45,945 vs 14,273 obstacle points). HPE should budget for the rich-scene case, not the median fixture.	Sections 15, 18

Nothing was fixed in D1, as instructed.

29. Files created / changed

Production source: **UNCHANGED.**

```
$ git diff --stat
(empty – no tracked file was modified)

$ git status --short
?? benchmarks/d1_execution/
```

Created (all untracked, all under `benchmarks/d1_execution/`):

File	Purpose
benchmarks/d1_execution/__init__.py	Package marker + scope statement
benchmarks/d1_execution/fixtures.py	Calibration, qualified config, stereo scenes, derived 2× calibration
benchmarks/d1_execution/measure.py	All 18 measurement sections + driver
benchmarks/d1_execution/results/d1_execution_audit.json	Raw machine-readable results
benchmarks/d1_execution/D1_EXECUTION_AUDIT_REPORT.md	This report

Not staged. Not committed. Not pushed. Not tagged.

Prior-session artefacts were moved to the session scratchpad before measurement began and are not part of the repository.

30. Final regression

Re-run after all benchmarks completed:

```
.venv/bin/python -m pytest -q
983 passed, 4 warnings in 19.89s
```

	Baseline (before)	Final (after)	Delta
passed	983	983	0
failed	0	0	0
skipped	0	0	0
errors	0	0	0
warnings	4	4	0
runtime	21.33 s	19.89 s	-1.44 s

Identical. The same 4 pre-existing `RuntimeWarning`s from `tests/test_d11_degradation_validation.py`'s deliberate NaN/Inf degradation tests. No regression, no new warning, no behavioural change.

31. Final recommendation

DPE is architecturally ready to serve as HPE's synchronous geometry provider, with one contract change required first.

What the audit confirms is sound:

- `process_geometry_frame(StereoObservation) -> GeometryFrame` is a genuine, verified single entry point. The dual-interface claim holds structurally, and the embedded path costs no more than the legacy one (30.94 ms embedded vs 31.13 ms legacy `process()`).
- DPE owns **no** execution policy. Zero queue, thread, async, or executor imports exist in production source. It is exactly the "provider, not orchestrator" shape the HPE architecture expects.
- Resource discipline is exemplary: construction is 0.157 ms and everything is built once. Over 400 frames, RSS was **flat at 178.4 MB** in steady state, latency drift was **-1.48 %** (decreasing), temporal history stayed pinned at its `max_records` bound, and all eight major objects retained identical identity.
- Performance is comfortable: **30.9 ms median ($\approx$ 32 FPS)** at the qualified 320×240, roughly **3.1× faster than the frozen NPE**. The full Level-4 temporal stack costs only **0.61 ms (1.9 %)** and is provably additive — raster geometry is byte-identical with temporal on and off.
- Drop tolerance is genuinely safe. Temporal history keys on timestamp alone, with no sequence number or arrival counter anywhere. **HPE may drop observations before submission for backpressure** — measured directly, with an identical state progression to a contiguous run — and DPE self-heals across large gaps via `ACCEPTED_NEW_SEQUENCE`.
- Failure semantics are usable: exceptions mean "drop this frame", returned frames are always structurally valid with degradation reported in-band through `quality.overall_state` and `degradation_reasons`.
- The calibration unit contract is unambiguous — millimetres at the `Q` boundary — and the anticipated Gazebo `baseline = 120.0` is **already correct**.

What must be decided before HPE async orchestration:

The **P0 observation-identity gap**. DPE's authoritative output carries no immutable per-observation correlator, so HPE's exact `GeometryFrame X` ↔ `SemanticFrame X` join cannot be expressed against the current contract. `GeometryFrame.frame_id` must not be repurposed — it carries real coordinate-frame semantics the entire nested evidence type graph depends on. DPE needs **two distinct concepts**: an immutable `observation_id` and the existing `coordinate_frame`. The input-side field (`StereoObservation.frame_id`) already exists and is simply inert, so the change is likely small — but it is a contract decision for the DPE/HPE boundary, not a D1 decision, and nothing was implemented.

What HPE should plan around:

Treat DPE as **one constructed instance, one active invocation at a time, non-decreasing timestamps, drops permitted, reordering not**. Concurrency between DPE and NPE is architecturally plausible but worth **at most 22.9 %** — NPE's 95.3 ms is the true system ceiling, and thread budgets for OpenCV and ONNX Runtime must be explicitly co-allocated on this 4-core machine rather than both defaulting to the full core count. Budget for the rich-scene case: DPE latency scales with how much valid geometry a scene yields, and raising resolution is dominated by the single-threaded geometry stages (6.0× cost for 4× the pixels), not by SGBM (3.6×).

Audit completeness. All 27 requested investigation areas were completed. The baseline suite passed 983/983 before and after. Production code is byte-for-byte unchanged. The one significant defect found is a contract gap, not a functional failure — and per the D1 pass/blocked rule, discovering an integration defect does not mean the audit failed.

D1 DPE SYNCHRONOUS EXECUTION AUDIT: PASS — READY FOR DPE/HPE CONTRACT DECISION